二维非线性泊松方程求解的 C++ 实现

芯钛量子 技术研发部

2025 年 9 月 18 日

Chapter 1

背景知识

1.1 泊松方程

许多物理问题都可以用偏微分方程（PDE）来描述，而**泊松方程**是其中之一，它具有以下形式：

$$\nabla \cdot (\nabla u) = f \quad \text{在 } \Omega \text{ 内} \tag{1.1}$$

简单起见，这里只考虑二维 PDE：即 Ω 是一个面域。u 是待求解的标量场。f 是一个已知的标量场，通常被称为源函数（source function）。

除了偏微分方程之外，许多物理问题的描述需要**边界条件**，这样的问题被称作边值问题。通常，边界条件分为两类：

- 狄利克雷边界条件（Dirichlet Boundary Condition），指定微分方程的解在边界处的值
- 诺伊曼边界条件（Neumann Boundary Condition），指定微分方程的解在边界处的微分

所有的边界条件构成区域 Ω 的外边界（常记作 $\partial\Omega$）。

1.2 非线性问题、牛顿迭代方法

如果 f 和 u 无关，那么方程1.1是线性的，否则是**非线性**的。

牛顿迭代是求方程根的重要方法。考虑方程 $F(u)=0$，将此方程在点 u_0 的某个邻域用泰勒公式展开，且只取其线性部分：

$$F(u) \approx \tilde{F}(u) = F(u_0) + \frac{\partial F}{\partial u}(u-u_0) = 0 \tag{1.2}$$

据此得出牛顿迭代法的一个关系式：

$$u_{n+1} = u_n - \frac{F(u_n)}{F'(u_n)} \tag{1.3}$$

1.3 有限单元法

在一般情况下，求出泊松方程的解析解是不可能完成的任务。因此，数值求解方法应运而生。**有限单元法**是一种常用的求解微分方程的数值方法。

将 Ω 进行网格剖分，划分为若干有限元，得到方程1.3的离散化形式：

$$\mathsf{A}(\mathbf{k}^{(e)})\delta\mathbf{u} = \mathsf{A}(\mathbf{f}^{(e)}) \tag{1.4}$$

A 是集成运算符（assembly operator），它代表我们要将 $\mathbf{k}$ 和 $\mathbf{f}$ 根据单元自由度表（Element Freedom Table）**叠加**到**全局刚度矩阵 $\mathbf{K}$** 和**全局荷载向量 $\mathbf{F}$** 中[1]。$\delta\mathbf{u}$ 是离散形式的迭代变化量。

$\mathbf{k}^{(e)}$ 是有限元的**单元刚度矩阵**，其中的第 (i,j) 分量为

$$k_{ij}^{(e)} = -\int_{\Omega_K} \nabla N_i \cdot \nabla N_j \mathrm{d}\Omega - \int_{\Omega_K} N_i N_j \frac{\partial f}{\partial u}\mathrm{d}\Omega \tag{1.5}$$

$\mathbf{f}^{(e)}$ 是有限元的**荷载向量**，其中的第 i 分量为

$$f_i^{(e)} = \int_{\Omega_K} \nabla N_i \cdot \nabla u \mathrm{d}\Omega + \int_{\Omega_K} f N_i \mathrm{d}\Omega \tag{1.6}$$

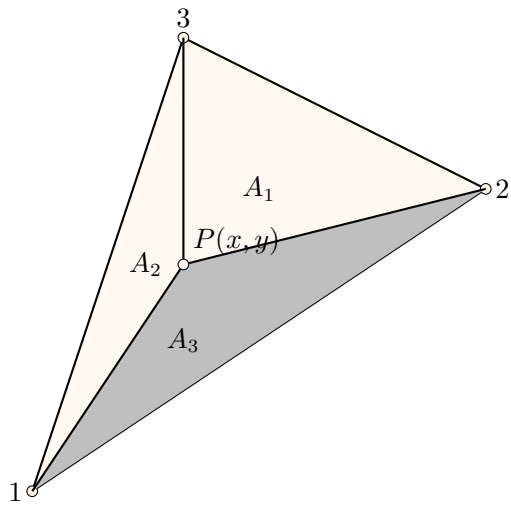
上面两式中的 N_i 是形函数（shape function），由单元的选取决定。

1.4 形函数

形函数可以理解为区域 Ω_K 中，u 的插值基函数。在二维问题中，我们经常采用一阶三角形单元或一阶四边形单元离散 Ω。

[1]集成过程体现了直接刚度法（Direct Stiffness Method）的思想。

1.4.1 一阶三角形单元



我们可以采用三角形的**面积坐标**来作为一阶三角形的形函数，其定义相当简单直接：

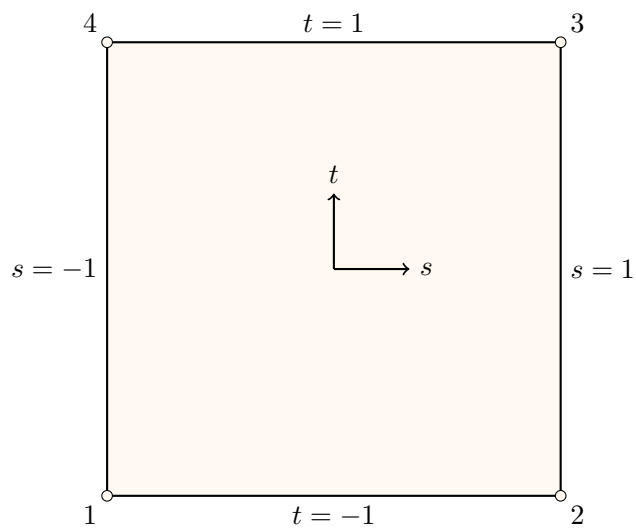
$$N_i = \frac{A_i}{A} \tag{1.7}$$

上式中，A_i 是三角形内某点 $P(x,y)$ 和 i 对边两顶点所形成的三角形的面积；A 是三角形单元的面积。

当然，我们也可以将形函数定义到等参空间之中，在计算数值积分时多做一次等参变换。

1.4.2 一阶四边形单元

由于直角坐标系下的四边形形状千变万化，且为了使得数值积分更加便利，一阶四边形单元的形函数定义在等参空间中：



形函数的构造需要符合要求：第 j 个节点代入第 i 个形函数的值为 Kronecker Delta，即 δ_{ij}。据此，可以构造出等参空间中，一阶四边形单元的形函数：

$$N_1 = \frac{1}{4}(1-s)(1-t) \tag{1.8a}$$

$$N_2 = \frac{1}{4}(1+s)(1-t) \tag{1.8b}$$

$$N_3 = \frac{1}{4}(1+s)(1+t) \tag{1.8c}$$

$$N_4 = \frac{1}{4}(1-s)(1+t) \tag{1.8d}$$

1.5 数值积分

由于被积函数中有源函数及其导数，因此，以解析方式计算1.5和1.6是非常烦琐的。此外，如果采用高阶单元，我们也无法得到上述积分的解析解。

在实际应用中,我们经常使用高斯-勒让德数值积分方法（Gauss-Legendre Quadrature）完成上述积分的求解：

$$\int_{-1}^{1} f(x)\mathrm{d}x \approx \sum_{k=1}^{n} f(\xi_k) w_k \tag{1.9}$$

上式中，ξ_k 是第 k 个高斯点的坐标（abscissa），w_k 是第 k 个高斯点的权重（weight），n 是高斯点的数目。

仅当函数 $f(x)$ 可以由在区间 $[-1,1]$ 上的多项式近似时，高斯求积才能获得准确的近似解。并且，有以下经过证明的事实：n 个高斯点可以精确计算出 $2n-1$ 阶多项式的积分。

在二维情况下（重积分），高斯求积方法扩展为

$$\int_{-1}^{1}\int_{-1}^{1} f(x,y)\mathrm{d}x\mathrm{d}y \approx \sum_{k=1}^{n}\sum_{l=1}^{m} f(\xi_k,\eta_l)w_k w_l \tag{1.10}$$

高斯积分点和权重通常被编制为表格[2]。

[2]如：https://pomax.github.io/bezierinfo/legendre-gauss.html

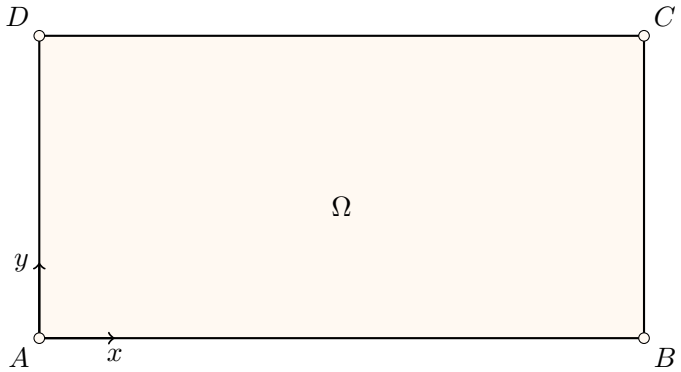
Chapter 2

问题描述、基本考察要求

您需要运用 `C++` 语言，使用一种或多种**程序设计技术**[1]，编写一个可以求解二维非线性泊松方程的程序。本章将更加详细地描述问题，并且对此程序的输入、输出和相应的算法文档作出要求。

2.1 求解区域

求解区域限定为一个如图所示的矩形：

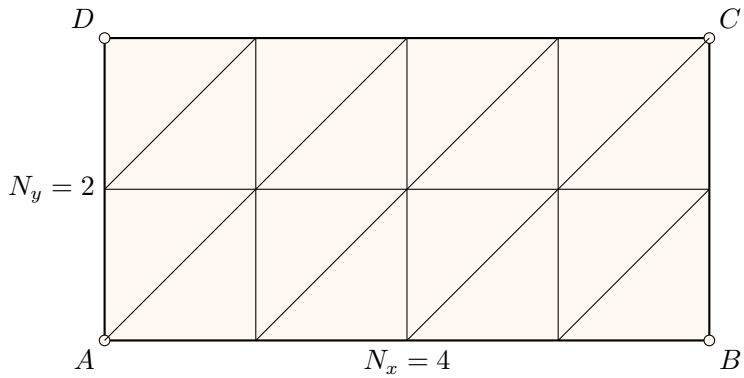


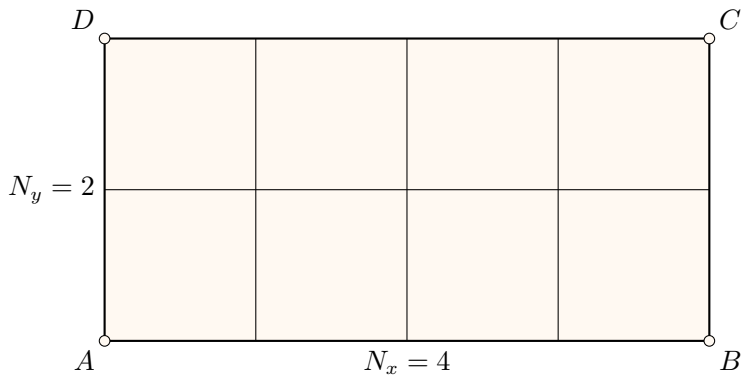
记矩形长为 l_x，宽为 l_y。用户会将这两个参数输入到您的程序之中。

[1] 如果您不知道何谓程序设计技术，请参考《C++ 程序设计语言》§1.2.1。

2.2 网格剖分

请把矩形的长边分为 N_x 段，宽边分为 N_y 段。据此，请将此区域全部剖分为一阶三角形单元，或者一阶四边形单元。如图所示：





同样地、N_x 和 N_y，以及单元类型（三角形/四边形）需要作为您程序的输入。

请留心网格（Mesh）的数据结构设计。您的设计至少需要正确地表示网格，节点（Node）和单元（Element）三者之间的关系。

2.3 边界条件

本次考核不考虑诺伊曼边界条件，只考虑狄利克雷边界条件：一个狄利克雷边界条件应该包括面域的某部分外边界 Γ_D，以及边界条件值 $\hat{u}$。

用户需要用某种方式定义狄利克雷边界条件的几何位置，您来设计用户定义边界条件的方式。边界条件的定义作为程序输入，且用户可能会输入不止一个边界条件。最重要的是，您需要计算出离散形式的**边界条件的索引**。

2.4 初始迭代值、源函数、源函数的导数

我们规定，初始迭代值 $\mathbf{u}_0$ 由一个函数 $u_0(x, y)$ 计算得到。

用户会以文本方式输入初始迭代值的计算函数 $u_0(x, y)$、源函数 $f(u)$、源函数的导数 $f'(u)$，您需要对用户输入的数学表达式作解析，并且您可以使用第三方库[2][3]。

2.5 单元项的计算

计算机无法理解公式1.5和1.6的具体含义，因为梯度算子和定积分对于计算机来说太复杂了。因此，请对上面两式做进一步的离散，使得梯度算子和定积分符号不再出现。如果有必要，请做相应的等参变换。请将您的推导整理为算法文档[4]。

2.6 单元项的叠加

1.3节没有详细介绍单元项的叠加过程。请您理解直接刚度法的思想，完成叠加方法（Assemblage）的算法文档。

2.7 狄利克雷边界条件的施加

第1章没有提及如何施加狄利克雷边界条件。请您调研狄利克雷边界的处理办法，并将其整理为算法文档。

[2] muParser：https://beltoforion.de/en/muparser/

[3] ExprTK：https://www.partow.net/programming/exprtk/index.html

[4] 在其中用自然语言、伪代码或流程图详细描述算法的步骤和逻辑，清晰地展示算法的执行过程和所需的输入输出。

2.8 线性代数库

用 `C++` 的标准模板库（`STL`）存储矩阵是非常痛苦的事情，更不必说还要实现线性代数的各类运算方法了。我们推荐您使用经过大量检验的第三方线性代数库，比如 `Eigen`[5]和 `Armadillo`[6]。您可以使用任何线性代数库完成此项考核。

2.9 牛顿迭代

假设现在进行到了第 n 步迭代，如果解向量和全局荷载向量同时满足以下条件，认为牛顿迭代收敛：

$$\frac{\|\delta\mathbf{u}^{(n)}\|}{\|\mathbf{u}^{(n)}\|} < \varepsilon_1 \tag{2.1a}$$

$$\|\mathbf{F}^{(n)}\| < \varepsilon_2 \tag{2.1b}$$

ε_1 和 ε_2 分别是**相对容差**和**绝对容差**。用户可以通过调整容差来指定求解的精度。

此外，用户需要输入**最大迭代次数**来防止迭代陷入死循环。

2.10 输入/输出

用户会将所有输入整合到一个配置文件中，其路径作为您程序主函数的入参。您可以在符合上述要求的前提下自由设计配置文件的格式。**请注意："读取配置文件"是一个很容易失败的操作。您需要在程序中处理可能发生的意外，以避免程序崩溃或者陷入未定义行为的泥沼。**

您的程序需要在控制台上打印牛顿迭代的流程，包括当前迭代步数，相对误差和绝对误差，以观察牛顿迭代的收敛情况。

您的程序需要将网格和解向量 $\mathbf{u}$ 输出到用户指定路径的文件。您需要使用某种方式看到计算结果（物理场）在 Ω 上的分布，并且看到计算结果在某条截线上的分布。我们推荐您使用 `VTK` 文件格式[7]存储计算结果，使用 `Paraview` 软件完成可视化。

[5] https://eigen.tuxfamily.org/index.php?title=Main_Page

[6] https://arma.sourceforge.net/

[7] https://docs.vtk.org/en/latest/design_documents/VTKFileFormats.htmlt

2.11 测试

如果设置狄利克雷边界条件 $u_{CD} = \hat{u}$，$u_{AB} = u_{BC} = u_{AD} = 0$，$f(u) = 0$，可以求出 $u(x, y)$ 的一个解析解：

$$u(x,y) = \frac{4\hat{u}}{\pi} \sum_{n=1,3,5,\ldots}^{\infty} \frac{\sin \frac{n\pi}{l_x} x \sinh \frac{n\pi}{l_x} y}{n \sinh \frac{n\pi}{l_x} l_y} \tag{2.2}$$

请说明您的计算结果和解析解吻合，并且分析数值解和解析解之间的误差。

2.12 例子

用户指定矩形的长为 2 个单位，宽为 1 个单位。长边分为 20 份，宽边分为 10 份。网格全部按照一阶四边形单元剖分。

用户向 AD 边施加狄利克雷边界条件，$u_{AD} = 2$；向 BC 边施加狄利克雷边界条件，$u_{BC} = 1$。

用户输入迭代初值计算函数是 $u_0(x, y) = e^{-x-y}$，源函数为 $f = e^{-u}$，源函数导数为 $\frac{\partial f}{\partial u} = -e^{-u}$。

用户设置相对容差和绝对容差均为 10^{-4}。当然，这个容差太大了。在这个容差下收敛的计算结果很可能不准确。

以下是一个可能的，由用户输入的任务文件：

Listing 2.1: input.txt

```
# 几何、网格信息
lx = 2.
ly = 1.
Nx = 20
Ny = 10
mesh = "rectangle"

# 边界条件信息
# ...

# 函数
```

```
guess = "exp(-x - y)"
source = "exp(-u)"
source_derivatives = "-exp(-u)"

# 牛顿迭代
rel_tol = 1e-4
abs_tol = 1e-4

# 输出文件所在路径
# ...
```

您可以自由扩充、修改任务文件的形式；也可以使用常见的数据交换格式，如 JSON 或者 XML。

任务文件的路径，会作为您程序 main 函数的参数。假设您的程序名为 Poisson2DSolver.exe，则应有如下调用：

```
Poisson2DSolver.exe input.txt
```

您的控制台程序需要在屏幕上打印迭代收敛的过程，下面是一种可能的做法：

```
-------------------------------------------------------------
Step      Abs. Error                    Rel. Error
-------------------------------------------------------------
0         3.45510262650798e-01          -
1         9.31829314875866e-02          1.34322907979504e-02
2         1.02227476413450e-02          1.55955568570741e-02
3         2.93189596366120e-03          5.75350782185927e-03
4         3.54605241422870e-04          1.33956488690905e-03
5         7.05769855308800e-06          1.37456980570782e-04
6         2.57976359291221e-09          2.19911977010614e-06
-------------------------------------------------------------
```

如果使用 Paraview 分析结果,您可以使用 Filter -> Plot Over Line 功能轻松获取 u 在某条线段的分布：

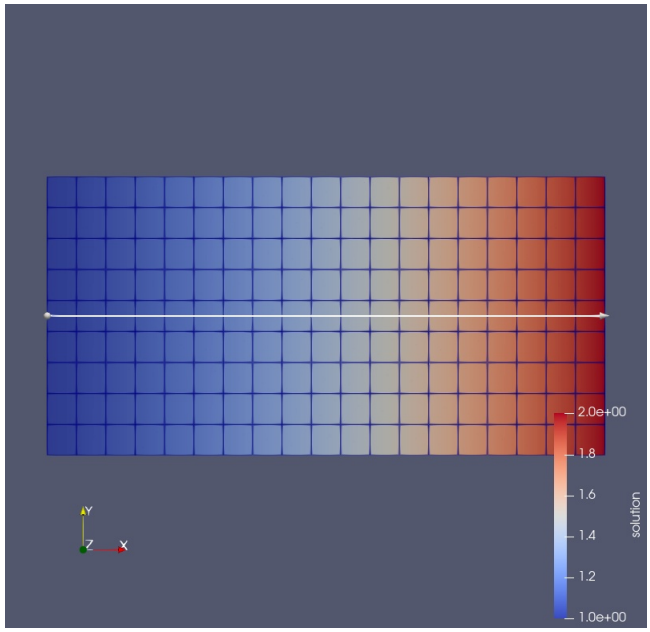


图 2.1: u 在 Ω 上的分布

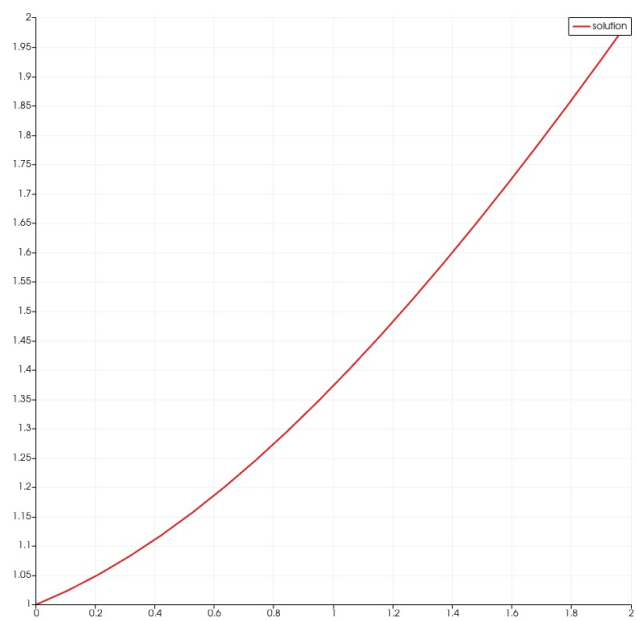


图 2.2: u 在 Ω 上沿长边的分布

Chapter 3

附加要求

1. 使用 `CMake` 构建您的 `C++` 项目。
   在大型项目的开发中，构建自动化工具是必不可少的。开发者使用它们处理编译、链接以至于各种库的依赖关系，也可以用它们拷贝文件、执行测试或者进行软件打包。`CMake` 拥有活跃的社区和丰富的文档资源，许多三方库都提供了 `CMake` 支持；此外，`CMake` 支持不同的操作系统、编译器、生成器，有着良好的跨平台支持。因此，我们要求您能够使用 `CMake` 构建自己的项目。

2. 使用稀疏矩阵。
   在全局刚度矩阵里，绝大多数元素都是零元素。这种性质被成为矩阵的**稀疏性**。如果我们只存储矩阵的非零元素，那么不仅能省下一笔非常可观的内存，而且由于许多算法只需要操作矩阵的非零元，我们还可以提升线性方程组求解算法的效率。**稀疏矩阵**[1]是这一问题的解决方案。
   请您分别用稠密矩阵和稀疏矩阵表示线性方程组，比较 N_x 和 N_y 分别为 20×10，100×50，200×100，1000×500 和 2000×1000 时，求解方程组所需的时间。请注意以下几点：

   - 使用脚本，自动化执行您的程序
   - 使用预处理器定义，控制编译时采用稀疏矩阵/稠密矩阵
   - 增加编译器的优化等级

[1] https://eigen.tuxfamily.org/dox/group___TutorialSparse.html

- **向稀疏矩阵中插入元素是非常低效的操作，请避免这样的行为**

3. 使用 `Boost` 库的高斯积分点和权重。
   `Boost` 可能是除了 `C++` 标准库之外，最重要的 `C++` 程序库了。很多 `Boost` 社群成员是 `C++` 标准委员会成员，这使得许多 `Boost` 子程序库会在未来变成 `C++` 标准库的一部分（如智能指针 `std::unique_ptr`，文件系统 `std::filesystem` 和函数对象 `std::function` 等）。此外，相较于相对贫瘠的 `C++` 标准库，`Boost` 无所不包，有很多值得发掘的地方。因此，无论如何，您都应该熟悉 `Boost`。

   作为“懒惰”的程序员，我们不喜欢重复的工作，比如，我们不喜欢将高斯点和权重的表格复制到代码中。一种解决方法是调用 `boost.math.quadrature.gauss` 模块[2]，获得高斯点坐标和权重。请注意，`Boost` 的接口只返回不小于零的高斯积分点和权重。

4. 将程序拆分到多个模块。
   工业级的应用具有庞大的代码规模。因此，如果不采用“库”的概念管理代码，您很快就会遇到以下问题：

   - 多名开发者同时编辑一个或几个巨大的源文件，导致代码难以编译通过，且代码审查极端困难
   - 编译缓慢，因为很小的改动可能会重新编译大多数代码
   - 针对不同业务的代码混在一起，开发者需要承受巨大的心智负担

   因此，在本次考核中，我们对程序设计做出这样的要求：您的程序应该由一个轻量的客户端，一个重量的库，和一个单元测试的可执行文件构成：

   - 客户端应该只包括一个文件和一个主函数，且这个文件不超过 100 行
   - 由于单元测试端需要链接这个重量级的库，因此，我们推荐您使用动态库（shared library）以控制生成的可执行文件的大小
   - 单元测试的具体细节会在下一个问题中介绍

[2] https://www.boost.org/doc/libs/1_85_0/libs/math/doc/html/math_toolkit/gauss.html

5. 使用测试框架。
   在软件开发中，我们需要**测试**来保证函数可以得到正确的计算结果。单元测试针对程序模块（软件设计的最小单位）来进行正确性检验的测试工作，它有以下的好处：

   - 通过单元测试的代码更加可靠
   - 如果修改了函数的实现，可以方便地检查新的实现的正确性
   - 合适的单元测试可以表达设计，并且督促开发者仔细设计自己的接口

   常用的单元测试框架有 `GoogleTest`[3]和 `Catch2`[4]。您可以使用其中之一完成2.11一节提到的测试。

6. 使用 `Doxygen`[5]自动生成代码文档。
   如果有客户使用您写的库，而且如果您没有撰写任何文档（documentation），那么，您一定会收到客户无休止的咨询或者抱怨。

   写文档可能是一件枯燥和麻烦的事情，您可能要在多份源文件之间游走。幸好，我们有一些**文档生成器**，可以让您以代码注释的方式写出美观便利的文档，`Doxygen` 就是其中之一。

   我们不会教您如何使用 `Doxygen`，请您自行学习，并且完成下列任务：

   - 给每个类写文档，至少包括 `@brief` 一项
   - 给每个类的 non-trivial 接口写文档，至少包括 `@brief`、`@param` 和 `@return` 三项
   - 给自己的项目配置 `Doxyfile`，并且生成 `HTML` 形式的帮助文档

7. 完成程序流程图和 UML 类图
   在成规模的开发过程中，算法的具体实现（如：某个矩阵的计算方法）往往是次要的，最重要的是程序的业务流程（如：先读取用户输入再做计算）。清楚完备的业务流程是良好的程序设计的必备条件，而流程图[6]是表示业务流程的最直观的方式。

[3] https://github.com/google/googletest
[4] https://github.com/catchorg/Catch2
[5] https://www.doxygen.nl/
[6] https://zh.wikipedia.org/wiki/%E6%B5%81%E7%A8%8B%E5%9B%BE

尽管没有限制程序设计技术，但是对于这样一个中等规模的项目来说，不使用 `class` 组织管理数据和方法是不现实的。随着类的引入，程序将很快地变得非常复杂。UML 类图[7]可以方便地表达设计，您需要掌握其基本知识以面对大型项目的开发。

请注意：

- **流程图、UML 类图和代码文档需要相容，不能出现矛盾。**
- **流程图重在表意，注重描述流程控制，尤其要注意处理程序的崩溃和出错问题。**
- **如果您是实习员工或者正式员工，请遵守《流程图编制规范》。**

[7] https://agilemodeling.com/artifacts/classDiagram.htm

Chapter 4

验收指标

4.1 基础指标

- ✓ 完成2.5，2.6和2.7节提到的算法文档，可以使用 LaTeX、更简单的 Markdown 或者类似的工具。
- ✓ 按照第2章要求设计完备的，可读性强的输入文件。
- ✓ 通过和解析解做对比，说明程序的计算结果是正确的。如果有必要，进行误差分析。

4.2 进阶指标

- ☐ 使用 `CMake` 构建项目。
- ☐ 使用一种单元测试框架，完成不少于两个单元测试。
- ☐ 将您的项目分为三个 `target`：一个轻量的客户端，一个重量的库，和一个单元测试的可执行文件。
- ☐ 使用 `Doxygen` 自动生成代码文档。
- ☐ 考虑用户的奇思妙想：防止程序在读配置文件时崩溃。
- ☐ 使用面向对象编程技术，简化处理一阶三角形单元和一阶四边形单元的逻辑。说明这样做如何提升程序的可扩展性。

✓ 使用稀疏矩阵存储全局刚度矩阵。比较不同规模网格下，采用稀疏矩阵和稠密矩阵对求解时间的影响。

4.3 高阶指标

- ☐ 推导1.5和1.6二式。
- ☐ 完成程序流程图和 UML 类图。
- ☐ 分析单步迭代过程的算法复杂度。
- ☐ 分析不同源函数/初始迭代值函数对牛顿迭代收敛性的影响。
- ☐ 使用有限体积法求解方程1.1，结果和有限元法一致即可。

4.4 数学考核指标

- ☐ 理解弱形式和原方程之间的关系。能够结合边界条件进行原方程到弱形式的推导。
- ☐ 理解网格元胞与形函数之间的联系。了解三角形元胞和四边形元胞上一阶形函数的标准表达式。
- ☐ 理解局部坐标系和标准坐标系之间的转换关系，并能对形函数进行相应的映射转换。
- ☐ 理解高斯积分的使用方法。
- ☐ 理解牛顿迭代法的基本原理，应用场景，和基本实现方法。
- ☐ 理解并实现单刚矩阵的构建过程，并思考构建过程中可预计算的部分。
- ☐ 理解并实现右手项的构建过程。
- ☐ 理解并实现单刚矩阵到总刚矩阵的组建过程，并思考组建过程中可加速的部分。
- ☐ 方程边界条件的种类和它们在求解编码过程中的体现方式。
- ☐ 思考结合牛顿迭代法的有限元求解实现方法。

- ☐ 思考结合含时算法的有限元求解实现方法，显式/隐式格式的差异和实现方法。
- ☐ 理解稀疏矩阵格式的意义，使用方法以及相应的稀疏矩阵线性系统求解器的应用。